Understanding the Language of Black Holes

Adviser:
Daniela Huppenkothen
NYU CDS
dh2288@nyu.edu

Nora Barry
NYU CDS
neb330@nyu.edu

Katrina Evtimova
NYU CDS
kve216@nyu.edu

Abstract

This work analyzes black hole X-ray emissions through the lens of language modeling. The authors propose a method to translate the time series into sequences and feed them into a Recurrent Neural Network (RNN) language model. Using the RNN states, these sequences are clustered to obtain a classification of black holes.

1 Introduction

In the field of astronomy, light curves represent the light intensity of a celestial object or region as a function of time. Based on the observations of changes in brightness over time, astronomers try to capture recurring patterns and identify different classes of stellar events such as the birth of a supernova. Within the existing literature, manual classification of light signals coming from black holes has been done in (Belloni et al., 2000). Also, deep learning has been applied to the supervised classification of supernovae (Charnock and Moss, 2016). However, in our case, we are provided with light curves data coming from black holes X-ray emissions with no pre-existing labels.

There are many reasons why scientists and astronomers may want a logical way to classify these emissions. For example, with a reasonable grouping strategy, one can dig deeper into specific categories and possibly discover patterns that were not previously recognized. Additionally, one may identify changes in a black hole's behavior if it begins producing emissions belonging to a different category than previous emissions.

The goal of this paper is to discuss a strategy for unsupervised clustering of black hole emissions that uses deep learning and natural language processing (NLP) strategies that have never before been applied to model the "language" of black holes. In general, as elaborated in (Längkvist et al., 2014), the task of unsupervised feature learning from time series data using deep learning is a challenging problem and and an active area of research. The authors of (Prasad and Prasad, 2014) discuss the use of RNNs for the task of time-series inference. Our strategy includes representing each time series as a sentence, and training a language model on this transformed data. In the end, we extract the outputs from each state in our RNN, and use these outputs as a new way of representing our data. The vector representations are then fed into a k-means clustering model, in which we are able form distinct clusters of time series emissions.

2 Existing Literature

Within the existing astronomical literature, (Belloni et al., 2000) classify different kinds of variability in black holes X-ray emissions coming from the Rossi X-ray Timing Explorer (RXTE) satellite. They analyze the emissions based on their light curves and color-color diagrams (CDs), detecting three main spectral states of the CDs denoted by A, B, and C. State A is associated with little time-variability and some red-noise variability, state B - with substantial red-noise variability, and state C - with white noise variability on a time scale of 1s. Belloni et al. (2000) produce a 12-class classification of the emissions depending on the pattern of transitions between these

states based on domain experience without the use of machine learning. Interestingly, the authors observe that certain patterns repeat at regular intervals of length months or years.

In our quest for methods that generate features from time series, we came across the Feature Extraction from Time Series paper from Harvard SEAS (Nun et al., 2015). The FATS Python library is built specifically for extracting over 60 potentially important features from astronomical light curve time data. Some of these features include mean, standard deviation, linear trend and skew from the input time series and error data. We use this feature extraction method as the baseline approach for clustering black holes data.

A fundamental approach to time series processing is presented in (LeCun and Bengio, 1995). The main focus of this paper is to showcase the successful applications of convolutions in different use cases, in particular for the analysis of time-series. Convolutions are a method of extracting and combining local features of the input data in order to produce higher-level representations. An example of a convolution is taking the mean or the maximum across a sliding window of neighboring values in a given input vector. The fact that time series data has one-dimensional structure with strong local correlations makes them good candidates for analysis using convolutional neural networks (CNN). Although this paper does not adopt this approach, it is worth noting that CNNs have instigated a remarkable progress in image and signal processing.

The Deep Embedding Clustering (DEC) (Xie et al., 2015) model uses deep learning to perform unsupervised classification of time series data by simultaneously learning feature representations and cluster assignments. In this two-stage setup, the data is first passed through a Stacked Autoencoder (SAE) which essentially de-noises the data and maps it into a feature space. Using k-means, the outputs from the SAE are then clustered and intermediate cluster centers are formed. The DEC model then uses the assigned clusters to update the embeddings by using stochastic gradient descent. The loss that is minimized in this case is the KL Divergence between the cluster assignments and a pre-defined target distribution from the current soft cluster assignment. This iterative process progressively improves the cluster assignments throughout training and is stopped once a small percentage of data points change cluster assignments in a certain iteration.

The current state of language modeling (LM) and its applications are studied in great depth in (Jozefowicz et al., 2016). The paper reveals many different tricks on training RNN LM models on large corpora of data such as regularization of big LSTM models through dropout, training with an approximation of Softmax using importance sampling (since its exact computation can be computationally expensive), and utilizing character-level CNN embeddings. The authors are aiming to inspire LM research at a large-scale.

The authors of (Dai and Le, 2015) present two approaches to unsupervised sequence learning with recurrent networks. The first one is a basic LSTM language model used for next step prediction in the conventional language modeling setting. The second one is an LSTM model initialized with a sequence autoencoder. The main idea of the latter is to use an RNN network as an encoder for an input sequence into a hidden state. The hidden state is then used as an input to a decoder recurrent network which predicts the output sequence. In our work, we incorporate this approach of using an LSTM RNN network as an autoencoder for extracting hidden states of a sequence at each processing step by the model.

3 Datasets

3.1 Astronomical Data

In terms of data, we use 13,496 time series of light emitted from 29 different black holes. The files are read from NASA satellites and encoded in FITS (Flexible Image Transport System) format. In addition to the light curve data, each file contains metadata such as the date and time of the observation and the telescope that was used. The frequency of the observations is at the second level and the series vary in terms of length between 15 seconds and 1000 seconds. The distribution of time series length can be seen in Figure 7 in the Appendix.

3.1.1 Pre-processing

The time series data are extracted using Python's library Astropy (Astropy Collaboration et al., 2013) which is specialized in processing FITS-formatted

data.

In order to transform the time series into a format that is conducive to deep learning architectures, we first consider using the methods included in (Nun et al., 2015). Using the authors' strategy, we are able to extract 59 features such as amplitude, autocorrelation and standard deviation for each time series. The results of clustering these FATS feature vectors can be seen in section 5.

Instead of using a single feature vector for a given time series, we consider applying FATS feature extraction to smaller subsets of the time series and thus obtain a sequence of FATS feature vectors corresponding to each segment. However, given the importance of data representation to this particular problem, we reason that FATS might not perform well on segments of a short length due to lack of enough information.

We explore alternative approaches for converting time series into sequential form and propose the following way of representing the data. After normalizing the time series, we split them into 32 and 64 second windows. It is worthy to note that the size of the interval at which we are splitting each time series is important since there might be periodicity present in the input data. The choice of window size should be based on our knowledge and observations of the input data. As a next step, we cluster these mini time series (amounting in total to more than 500,000 for all time series) into 10,000 different clusters for both window sizes. Here, the number of clusters can be thought of as a vocabulary size, with each cluster assignment corresponding to an individual word in our vocabulary. As a result, every time series is represented as a vector of cluster assignments (or words). We further standardize each vector representation of time series to be of length 100 through padding and slicing. With this sequence representation, we aim to we fit a language model to predict the next "word" in each time series.

3.2 Text Data

In addition to astronomical data, we decide to evaluate our language modeling methodology on easier-to-interpret, real text data. We use the Rotten Tomatoes dataset from Assignment 1 consisting of 10,662 sample reviews belonging to one of the two categories: positive and negative. Standard preprocessing techniques are applied to the data. We randomly shuffle and split the data into 9602 training and 1062 test samples. Following the same logic as for the time series data, we standardize each review to be of size 100 by cutting longer ones or padding shorter ones with the "⟨unk⟩" token.

4 Model

We adapt and modify the RNN language modeling technique from (Zaremba et al., 2014) and apply it in the particular task of unsupervised clustering of sequences. We build our implementation using code from (TensorFlow, 2015). Using the time series sequence representations of length 100 as input to the RNN, we fit the RNN to the data, setting the number of steps to 100 for each batch. The model architecture includes 2 hidden LSTM layers of size 200 each. We use a stochastic gradient descent optimizer with learning rate 0.1, no dropout, and a batch size of 20. In this setting, we seek to minimize the average negative log probability of the target words, or, alternatively, maximize the probability of the target words.

The main idea is to utilize the hidden states of the RNN model for each time series. In particular, at the end of the model fitting process, for a given sequence corresponding to a time series, we average over the outputs from each step in the RNN, and use this vector of length 200 as our representation learned by the RNN language model. The end goal is to feed these vectors into yet another k-means model, and visualize the clusters using tSNE. Our results are presented in the following section.

A possible modification of our current methodology is to use the actual sentence length instead of forcing a fixed input sentence length of 100. In particular, cutting longer sentences or padding shorter ones might be introducing features not present in the original data and thus hindering our results. We have a working implementation of this alternative approach but are yet to investigate the outputs from it.

5 Results & Analysis

5.1 Baseline: Clustering on FATS features

Feeding the FATS features for each time series into a baseline k-means model and using t-distributed

stochastic neighbor embedding (tSNE), we are able to visualize at least two distinct clusters.

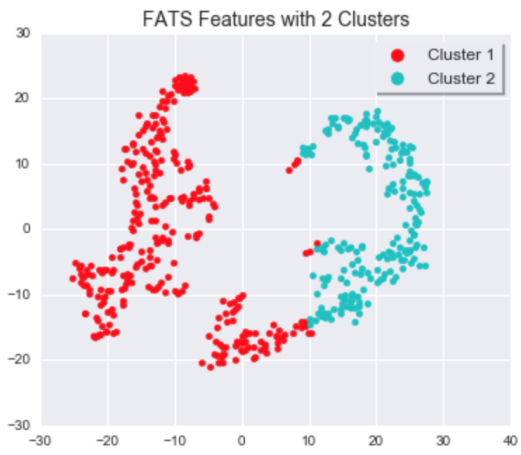

Figure 1: Baseline Clustering of FATS Vectors

Visualizing sample time series from each of these clusters in a similar fashion to what is done in Figure 5 might be helpful in understanding whether the time series in each cluster are "semantically" different and requires domain knowledge of the data.

5.2 Clustering on Sequence Representations

We apply k-means clustering to the RNN representations of sequences obtained from time series segments of length 32 and 64 seconds, respectively. In the end, find that the 32-windows experiment yields better results. Analysis of the 32-windows experiment follows bellow. Visualizations of the 64-windows experiment can be found in the Appendix.

The training and validation loss is shown in Figure 2. We also report the training and validation perplexity throughout the modeling process to monitor overfitting. Interestingly, the validation perplexity never exceeds the training perplexity, which is shown in Figure 3. We believe this is due to the fact that our transformed time series data is unlike true sentences in at least one way: there are many repeated words in the time series, which occurs when adjacent windows are assigned to the same cluster. This is due to the fact that often there is minimal change in the magnitude of the time series, so adjacent windows are actually very similar and hence clustered together. As a result, the model is able to recognize this trend from the training data, and successfully apply it to a similar, much smaller validation dataset. It is also very likely that the behavior of these time series is more predictable than natural language. After 10 epochs, the we are able to reach a training perplexity of 22.38 and validation perplexity of 14.57.

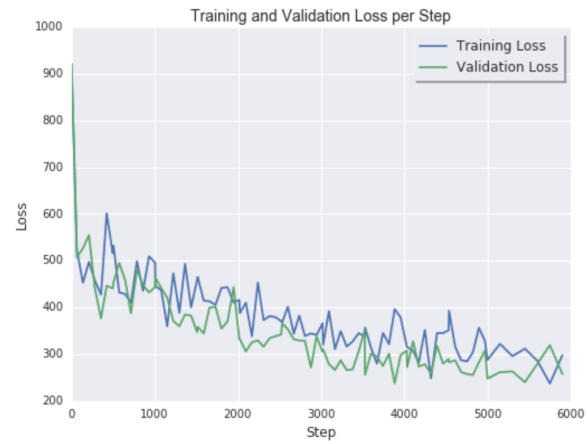

Figure 2: Training and Validation Loss

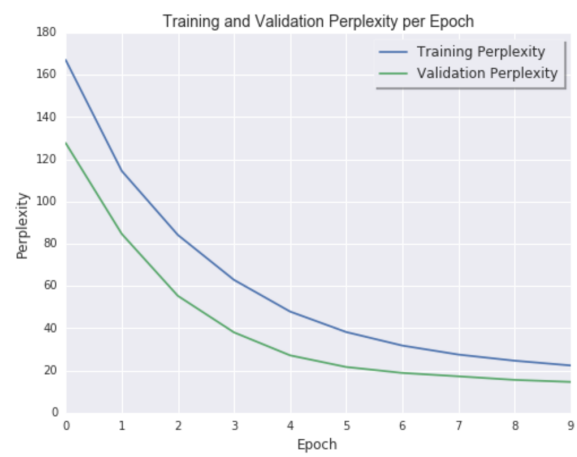

Figure 3: Training and Validation Perplexity

Figures 4 shows the result of clustering the outputs for 32-window time series into two clusters. Visualizations for three, four and five clusters can be found in the Appendix. The same cluster sizes for the 64 windows are also listed in the Appendix but are less interpretable than the 32-window ones.

With unsupervised clustering, it is often unclear how many clusters to choose. In our case, the manual classification of the black hole emissions in (Belloni et al., 2000) yielded 12 distinct groups. In addition, the inertia, or minimizing criterion of k-means, forms an "elbow" at around 8 clusters. However, it

Figure 4: tSNE with 2 clusters

becomes hard to validate a large number of clusters visually. However, by plotting a random sample of time series that fall into each cluster, we are able to see that at least some of the groupings visually make sense. This is demonstrated in Figure 5. For example, in cluster one and cluster eight we see a difference in the type of time series, as those in cluster eight are more mean-reverting. This implies that the RNN representations of the time series are able to discriminate between different features of the underlying time series.

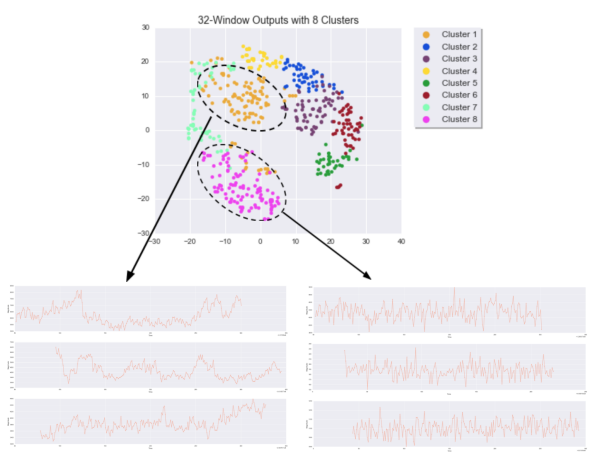

Figure 5: tSNE on 32-window data with 8 clusters and time series examples from two clusters

A non-trivial aspect of our unsupervised learning task is finding a suitable evaluation metric. Many times, the "ground truth" can be used to evaluate a model, ie. using a subset of labeled data during testing. However, since black hole emissions have never been scientifically classified at a large scale, we are unable to manually label our data. In fact, our model is one of the first attempts at unbiasedly labeling this data. In the following section we seek to evaluate our model by testing its performance on actual text data, to see if it is able to generate accurate representations of language that can then be clustered with k-means.

5.3 Clustering on Rotten Tomatoes Data

We retrain and apply our model to the movie review data from Rotten Tomatoes (Kaggle, 2014), (Socher et al., 2013), and extract vector representations from the RNN autoencoder for each movie review. If our model is doing what it is supposed to, we should be able to cluster the vector representations into two distinct groups, one corresponding to negative reviews and one to positive reviews. The result of this experiment is shown in Figure 6. It is evident that there are two clusters.

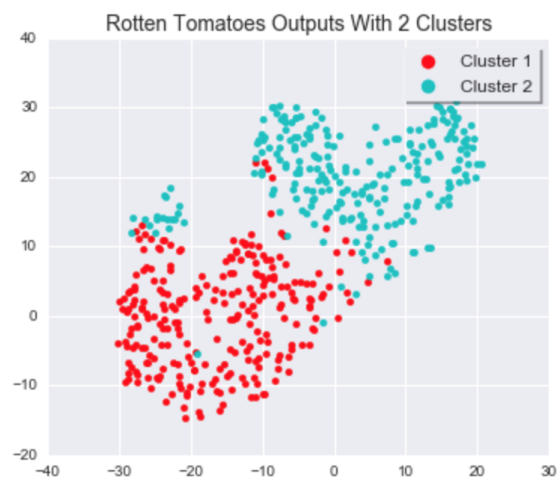

Figure 6: tSNE on Rotten Tomatoes data with 2 clusters

Analysis of the classification shows that the polarity distribution of positive to negative reviews is pretty even for both of the clusters (49% negative in the first and 51% negative in the second), meaning that sentiment is not the deciding factor in cluster formation. This result could likely be improved with more sophisticated data cleaning strategies and hyperparameter tuning or the introduction of regular-

ization to our model, for example, through the use of dropout as suggested in (Zaremba et al., 2014). However, further analysis of the clusters may reveal some other intrinsic feature or combination of features of the data which plays a role in the cluster assignment. A possible future direction for our project is investigating which the splitting factor for the RNN representations is in the case of the Rotten Tomatoes reviews in order to understand better how the model works.

6 Conclusion

This work presents a novel approach to analyzing black hole time series using techniques from deep learning and natural language understanding for the task of black hole classification. The authors propose a way of transforming the time series data into sequence representations which serve as input to an RNN LSTM language model. Using the hidden states from the RNN, the authors are able to extract new representations of the input sequences and cluster them using k-means. They validate their approach on actual text data. This work is one of the first steps in bridging the gap between black hole research and deep learning and can be used as a starting point for future work in this area. In addition, techniques from works such as (Dai and Le, 2015) and (Jozefowicz et al., 2016) can be especially useful for future research in this domain.

Acknowledgements

We are very grateful to our adviser Daniela Huppenkothen for introducing us to this interesting problem and for her ongoing advice and support. We would also like to thank professors Sam Bowman, Kyunghyun Cho, and Carlos Fernandez-Granda as well as Andrew Drozdov and the course TAs Tian Weng and Meihao Chen for the helpful discussions.

Collaboration Statement

Both authors did literature review and discussed extensively the modeling strategy and architecture. Nora worked on implementing the model in TensorFlow and creating the visualizations. Katrina worked on data pre-processing, adapting and running the model on the CIMS machines. Both authors contributed equally to the write-up. Our code can be found in: `https://github.com/neb330/BlackHoles`.

References

Astropy Collaboration, T. P. Robitaille, E. J. Tollerud, P. Greenfield, M. Droettboom, E. Bray, T. Aldcroft, M. Davis, A. Ginsburg, A. M. Price-Whelan, W. E. Kerzendorf, A. Conley, N. Crighton, K. Barbary, D. Muna, H. Ferguson, F. Grollier, M. M. Parikh, P. H. Nair, H. M. Unther, C. Deil, J. Woillez, S. Conseil, R. Kramer, J. E. H. Turner, L. Singer, R. Fox, B. A. Weaver, V. Zabalza, Z. I. Edwards, K. Azalee Bostroem, D. J. Burke, A. R. Casey, S. M. Crawford, N. Dencheva, J. Ely, T. Jenness, K. Labrie, P. L. Lim, F. Pierfederici, A. Pontzen, A. Ptak, B. Refsdal, M. Servillat, and O. Streicher. 2013. Astropy: A community python package for astronomy. *arXiv*, 558:A33, October.

T Belloni, M Klein-Wolt, M Méndez, M Van Der Klis, and J Van Paradijs. 2000. A model-independent analysis of the variability of grs 1915+ 105. *arXiv preprint astro-ph/0001103*.

Tom Charnock and Adam Moss. 2016. Deep recurrent neural networks for supernovae classification. *arXiv preprint arXiv:1606.07442*.

Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in Neural Information Processing Systems*, pages 3079–3087.

Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 2016. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*.

Kaggle. 2014. Classify the sentiment of sentences from the rotten tomatoes dataset. *https://www.kaggle.com/c/sentiment-analysis-on-movie-reviews*.

Martin Längkvist, Lars Karlsson, and Amy Loutfi. 2014. A review of unsupervised feature learning and deep learning for time-series modeling. *Pattern Recognition Letters*, 42:11–24.

Yann LeCun and Yoshua Bengio. 1995. Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*, 3361(10):1995.

I. Nun, P. Protopapas, B. Sim, M. Zhu, R. Dave, N. Castro, and K. Pichara. 2015. FATS: Feature Analysis for Time Series. *ArXiv e-prints*, May.

Sharat C Prasad and Piyush Prasad. 2014. Deep recurrent neural networks for time series prediction. *arXiv preprint arXiv:1407.5949*.

Richard Socher, Alex Perelygin, Jean Y Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and

Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the conference on empirical methods in natural language processing (EMNLP)*, volume 1631, page 1642. Citeseer.

TensorFlow. 2015. TensorFlow RNN tutorial. *https://www.tensorflow.org/tutorials/recurrent/*.

Junyuan Xie, Ross Girshick, and Ali Farhadi. 2015. Unsupervised deep embedding for clustering analysis. *arXiv preprint arXiv:1511.06335*.

Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. 2014. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*.

Appendix

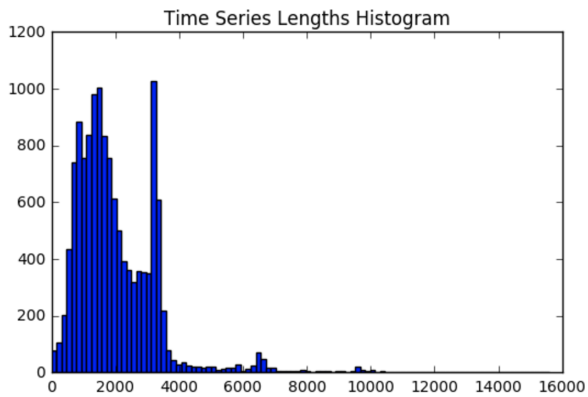

Figure 7: Time Series Length (seconds) Distribution

Figure 8: tSNE on 32-window data with 3 clusters

Figure 9: tSNE on 32-window data with 4 clusters

Figure 10: tSNE on 32-window data with 5 clusters

Figure 11: tSNE on 64-window data with 2 clusters

Figure 12: tSNE on 64-window data with 3 clusters

Figure 13: tSNE on 64-window data with 4 clusters

Figure 14: tSNE on 64-window data with 5 clusters